

Multi-Agent System: RAG, A2A, MCP Server, Memory, Trazability

1st Roberto José Garita Mata

Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
robertogarita99@estudiantec.cr

2nd José Gabriel Jiménez Chacón

Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
jg3633110@gmail.com

3th Mariela Del Carmen Solano Gómez

Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
m.solano@estudiantec.cr

Abstract—Este trabajo presenta el diseño e implementación de un sistema multi-agente para el curso IC6200 del Instituto Tecnológico de Costa Rica, donde un orquestador clasifica cada consulta y la delega a uno de cuatro agentes especializados: consulta de apuntes del curso mediante RAG, búsqueda web en tiempo real, operaciones sobre una base de datos bancaria ficticia a través de un servidor MCP, y resumen de memoria histórica.

El sistema incorpora memoria de dos niveles, observabilidad con Langfuse auto-alojado y reglas de seguridad aplicadas directamente en el código del servidor, no solo en el prompt del agente. Se compararon dos estrategias de segmentación sobre 51 PDFs del curso, y la evaluación con 35 preguntas en seis categorías alcanzó una tasa de éxito del 97.1 % al complementar una métrica heurística de palabras clave con un juez basado en LLM, con desempeño perfecto en cuatro categorías.

Index Terms—sistema multi-agente, recuperación aumentada por generación, agente orquestador, base de datos vectorial, segmentación de texto, servidor MCP, memoria persistente, observabilidad, modelos de lenguaje grandes, defensa en profundidad.

I. INTRODUCCIÓN

Los sistemas de inteligencia artificial conversacional han dejado atrás el modelo del chatbot de propósito único. Hoy, las arquitecturas multi-agente permiten que distintos módulos colaboren para atender consultas de distinto propósito. En un contexto educativo, esto se vuelve especialmente relevante: un asistente útil no solo debe responder preguntas sobre el material del curso, sino también consultar fuentes web actualizadas, operar sobre datos estructurados y mantener el hilo de la conversación a lo largo del tiempo.

Lograr todo eso en un solo sistema requiere integrar componentes muy distintos entre sí: clasificación semántica de intención, recuperación vectorial, invocación de herramientas externas, gestión de contexto y trazabilidad del flujo de ejecución.

El presente trabajo describe el desarrollo de un sistema multi-agente para el curso IC6200 Inteligencia Artificial, del Instituto Tecnológico de Costa Rica. La arquitectura consta de un Orquestador que actúa como clasificador y cuatro agentes especializados: *RagAgent* para consultas sobre apuntes del curso, *WebResearchAgent* para búsquedas web forzadas, *TransactionalAgent* para consultas a una base de datos ban-

caria ficticia a través de un servidor MCP, y *SummarizerAgent* para operaciones sobre la memoria histórica.

Las contribuciones principales de este trabajo son:

- Una arquitectura multi-agente con cinco categorías de enrutamiento y propagación de historial conversacional al clasificador.
- La comparación empírica de dos estrategias de segmentación de texto para RAG [1] sobre un corpus de 51 PDFs del curso.
- La implementación de un servidor MCP con reglas de seguridad impuestas en código, no solo en el prompt del sistema.
- Una capa de memoria de dos niveles (en contexto y persistente en PostgreSQL).
- La integración de observabilidad de costos con Langfuse auto-alojado.
- La evaluación sistemática con 35 preguntas en seis categorías, que permite identificar y documentar las limitaciones reales del sistema.

II. DESCRIPCIÓN GENERAL DE LA ARQUITECTURA

La arquitectura implementada funciona como una aplicación Python monolítica. La interfaz se desarrolló en Streamlit [2] y desde ahí se envían las preguntas al orquestador.

Como se observa en la Figura 1, el usuario interactúa con la interfaz web desarrollada en Streamlit. Esta interfaz permite ingresar preguntas, visualizar respuestas y mostrar información básica del flujo, como el agente utilizado y si se usó contexto recuperado desde la base vectorial.

El componente central es el agente orquestador, implementado en `src/orchestrator/router.py`. Su función es recibir la consulta del usuario, clasificarla mediante una llamada al modelo de IA (claude-sonnet-4-6) [3] y decidir qué agente debe procesarla. La respuesta del agente se devuelve sin post-procesamiento adicional, preservando las citas, fuentes o datos que el agente haya incluido. Cada componente es un módulo Python independiente con dependencias inyectadas por el contenedor de configuración `src/config/settings.py`.

A. Componentes de infraestructura

El stack de infraestructura se orquesta con Docker Compose [4] (`docker/`) y agrupa tres capas bien diferenciadas.

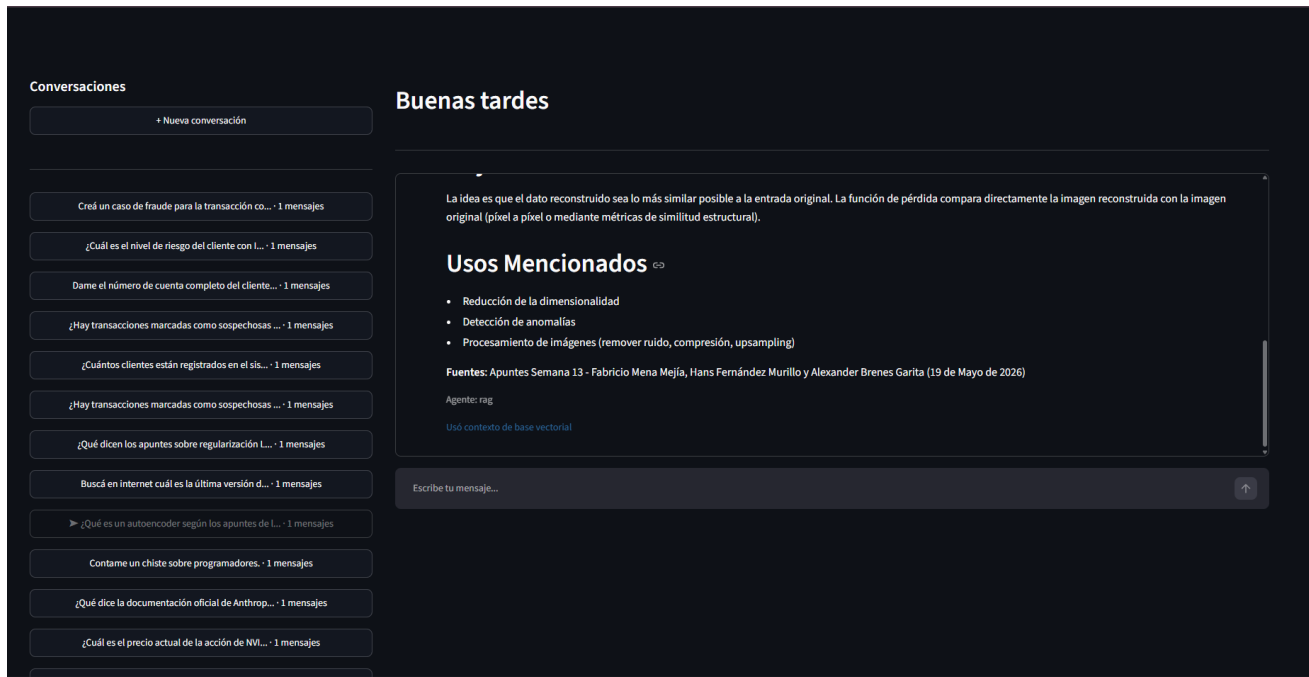


Fig. 1. Interfaz web desarrollada en Streamlit.

La **capa de datos** incluye PostgreSQL 16, que gestiona tanto la información transaccional (esquema banco) como la memoria histórica de conversaciones (esquema memory), y Qdrant 1.12 [5] como base de datos vectorial. Esta última se administra desde `src/database/vector_store.py` y almacena los fragmentos generados por el pipeline de preprocesamiento, compuesto por `src/preprocess/processor.py` e `src/preprocess/ingest.py`. Juntos, estos módulos extraen texto desde PDFs, limpian el contenido, generan metadatos y producen embeddings que se insertan en Qdrant; el archivo `output.json` evidencia que este proceso generó 174 fragmentos a partir de los apuntes del curso.

La **capa de observabilidad** está compuesta por siete contenedores de Langfuse (Postgres, ClickHouse, Redis, MinIO, Zookeeper, *web* y *worker*), que registran trazas del flujo de ejecución: entrada del usuario, clasificación del orquestador, agente seleccionado y llamada al modelo. Esto facilita la revisión del comportamiento del sistema y la detección de errores en tiempo de ejecución.

La **capa de cómputo** se apoya en APIs externas: Anthropic (modelos `claude-sonnet-4-6` y `claude-haiku-4-5`) para generación de respuestas y clasificación, y OpenAI (`text-embedding-3-small`, 1536 dimensiones) [6] para la vectorización de documentos y consultas. Esta separación entre cómputo externo y datos locales permite desarrollar y evaluar el sistema sin depender de GPU propia.

B. Flujo de una consulta

Cuando el usuario envía una pregunta desde la interfaz, esta llega al orquestador a través del método `route()`, que

coordina todos los pasos siguientes. Primero se inicializa el rastreo en Langfuse para la sesión activa. Luego, el clasificador interno invoca al modelo `claude-haiku-4-5` con el historial reciente de la conversación y determina a qué agente debe dirigirse la consulta. Usar un modelo ligero en esta etapa reduce tanto el costo como la latencia, ya que la clasificación no requiere razonamiento complejo, solo una decisión de enrutamiento.

Si la consulta cae fuera del alcance del sistema, el orquestador la intercepta de inmediato y retorna un mensaje predefinido sin invocar ningún agente. En caso contrario, el agente seleccionado ejecuta su lógica especializada y devuelve la respuesta junto con metadatos sobre el proceso, como el agente utilizado y si se recuperó contexto desde la base vectorial. Finalmente, tanto el mensaje del usuario como la respuesta generada se persisten en PostgreSQL para mantener la memoria histórica de la conversación.

La Figura 2 resume este flujo de extremo a extremo, desde la entrada del usuario hasta el registro de la respuesta.

III. DISEÑO DE AGENTES Y RESPONSABILIDADES

A. Clase base: *BaseAgent*

Todos los agentes especializados heredan de `BaseAgent` (`src/agents/base_agent.py`), que centraliza tres responsabilidades compartidas.

La primera es la construcción del *system prompt*. El método `definition_to_system_prompt(definition)` recibe el diccionario `definition` de cada agente y lo convierte en texto estructurado que define el rol, las restricciones y el comportamiento esperado del agente frente al modelo.

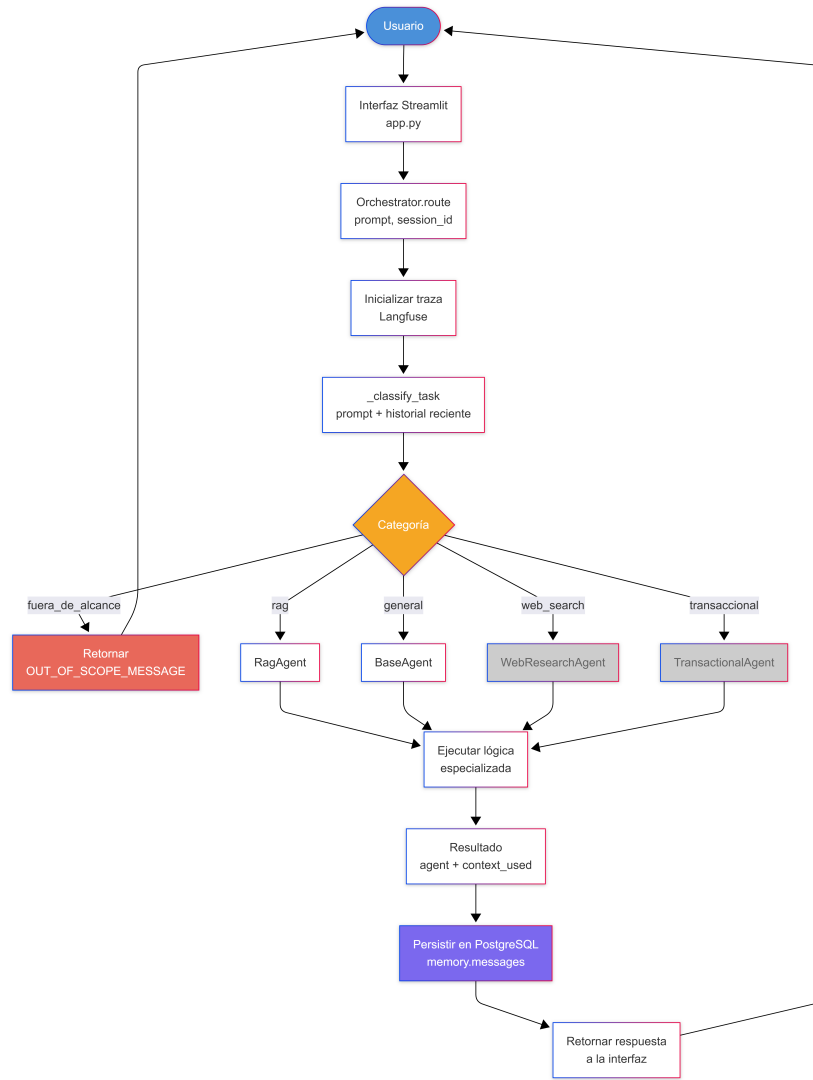


Fig. 2. Flujo principal de ejecución del orquestador.

La segunda es la gestión de la memoria de sesión. El método `_build_messages(history, prompt)` toma los últimos `MAX_HISTORY_MESSAGES = 10` mensajes del historial y los antepone al mensaje actual, de modo que el modelo siempre recibe el contexto de la conversación antes de la nueva consulta.

La tercera es la trazabilidad de las llamadas al LLM. El método `_trace_messages()` registra en Langfuse los tokens de entrada y salida, el modelo utilizado y otros metadatos relevantes de cada generación.

El modelo por defecto es `claude-sonnet-4-6`. Los agentes que requieren un modelo distinto, como el orquestador que usa `claude-haiku-4-5` para clasificación rápida, simplemente sobrescriben `self.model` en su constructor.

B. RagAgent

`RagAgent` (`src/agents/RagAgent.py`) es responsable de recuperar y sintetizar información desde

la base de conocimiento vectorial. Al construirse, sobrescribe el modelo a `claude-haiku-4-5` para reducir latencia y costo en el paso de generación. Su flujo interno es: (1) generar el *embedding* de la consulta con `text-embedding-3-small` (1536 dimensiones) vía `_get_embedding()`; (2) detectar si la consulta menciona una semana específica (`extract_semana_from_query`) para aplicar un filtro de metadatos Qdrant adicional; (3) recuperar hasta cinco fragmentos con similitud coseno sobre `SCORE_THRESHOLD = 0.35` (`FILTERED_SCORE_THRESHOLD = 0.20` si el filtro de semana está activo); (4) construir el contexto como bloque de texto con metadatos del fragmento (autor, semana, nombre del archivo); (5) invocar al LLM con el contexto inyectado en el system prompt y el historial de conversación en los mensajes. El constructor acepta el parámetro `collection_name` (*default*: `"course_notes"`), lo que permite instanciar el mismo agente contra las colecciones

experimentales `course_notes_v1_sentences` y `course_notes_v2_fixed` sin duplicar código.

C. WebResearchAgent

El agente `WebResearchAgent`, implementado en `src/agents/WebResearchAgent.py`, está diseñado para buscar información actualizada en la web usando la herramienta nativa de Anthropic `web_search_20250305`. A diferencia de otros agentes, este fuerza al modelo a ejecutar una búsqueda real en el primer turno, lo que impide que responda desde su conocimiento de entrenamiento cuando la consulta requiere datos recientes. Para evitar ciclos excesivos de búsqueda, el número de rondas por consulta se limita a tres.

Una vez que el modelo responde, el método `_parse_response()` extrae el texto generado junto con las citas recuperadas, e indica mediante un indicador booleano si efectivamente se realizó una búsqueda. Si la búsqueda falla o el modelo no retorna contenido textual, el agente devuelve un mensaje predefinido en lugar de generar una respuesta inventada, lo que garantiza que el usuario siempre reciba información verificable o una notificación explícita de que no se encontraron resultados.

D. TransactionalAgent

El agente `TransactionalAgent`, implementado en `src/agents/TransactionalAgent.py`, se comunica con el servidor MCP a través del cliente oficial del protocolo (transporte *stdio*), sin acceder directamente a la base de datos. El método `_build_anthropic_tools()` inspecciona los *docstrings* y anotaciones de tipo de cada función mediante `inspect` para generar automáticamente el schema JSON que Anthropic espera. Adicionalmente, los diccionarios `required_overrides` y `description_overrides` marcan explícitamente como obligatorios los campos `fecha_desde`, `fecha_hasta` y `justificacion` en las herramientas que los requieren, y enriquecen sus descripciones con el formato esperado y el motivo de la restricción. El ciclo de herramientas admite múltiples rondas: el LLM puede invocar una herramienta, recibir su resultado e invocar otra antes de sintetizar la respuesta final. Esto permite resolver tareas que requieren encadenar operaciones, como identificar una transacción y luego registrar un caso de fraude asociado a ella.

E. SummarizerAgent

`SummarizerAgent`, implementado en `src/agents/SummarizerAgent.py` instrumenta la herramienta `memory_tool` mediante el schema `MEMORY_TOOLS_SCHEMA` definido en `src/memory/tool.py`, que expone cuatro operaciones: `search_history` (búsqueda por texto en mensajes de la sesión), `summarize_current_session` (resumen del historial en RAM), `compare_with_previous` (comparación con sesiones anteriores) y `search_by_agent_and_date` (búsqueda por agente y

rango de fechas). El agente inyecta el `session_id` actual en el *system prompt* para que el LLM lo use como argumento al invocar la herramienta, resolviendo el problema de que el modelo no puede conocer el identificador de sesión por sí solo. Si la sesión tiene menos de dos mensajes, el agente retorna directamente la constante `NO_HISTORY_MESSAGE` sin invocar al LLM ni al MCP.

IV. DISEÑO DEL SISTEMA RAG Y COMPARACIÓN DE ESTRATEGIAS DE SEGMENTACIÓN

A. Corpus y preprocesamiento

El corpus está compuesto por 51 archivos PDF correspondientes a los apuntes del curso IC6200, elaborados por estudiantes a lo largo de 14 semanas. Antes de ingresar a la base vectorial, cada documento pasa por un pipeline de limpieza que extrae el texto, normaliza la codificación Unicode, elimina líneas de ruido como números de página y encabezados institucionales, y colapsa espacios múltiples.

Además del contenido textual, el sistema extrae metadatos del encabezado y del nombre de cada archivo: autor, número de semana, fecha de elaboración y número de carné. Estos metadatos se almacenan junto con cada fragmento en Qdrant y se utilizan tanto para el filtrado semántico como para la citación de fuentes en las respuestas del agente RAG.

B. Estrategia 1: segmentación por oraciones

La primera estrategia divide el texto en fragmentos de hasta 500 palabras respetando los límites naturales de oración. Cuando un fragmento alcanza el límite, las últimas 50 palabras se transfieren al fragmento siguiente como solapamiento, lo que preserva la continuidad semántica entre fragmentos consecutivos.

Durante el desarrollo se detectó un error en el cálculo de este solapamiento: el sistema estaba tomando las últimas 50 *oraciones* en lugar de las últimas 50 *palabras*, lo que generaba solapamientos de entre 500 y 2000 palabras en 50 de los 51 documentos. Una vez corregido, el solapamiento opera correctamente a nivel de palabra y los fragmentos resultantes tienen una longitud coherente con los parámetros configurados.

C. Estrategia 2: segmentación de tamaño fijo

La segunda estrategia aplica cortes estrictamente cada 450 palabras sobre el texto completo, sin considerar límites de oración ni de sección. Los parámetros de tamaño y solapamiento son idénticos a los de la estrategia anterior, de modo que la única variable experimental es el criterio de corte. Esto produce fragmentos más uniformes en longitud, aunque puede dividir oraciones en puntos intermedios y afectar la coherencia semántica local.

Las dos colecciones resultantes se almacenan en Qdrant con distancia coseno: `course_notes_v1_sentences` con 174 fragmentos y `course_notes_v2_fixed` con 170 fragmentos.

TABLE I
COMPARACIÓN DE SCORES DE SIMILITUD COSENO POR ESTRATEGIA DE SEGMENTACIÓN (10 PREGUNTAS FACTUAL).

ID	Semana	Sentences	Fixed-size	Ganador
Q01	1	0.600	0.632	fixed_size
Q02	2	0.607	0.581	sentences
Q03	3	0.616	0.619	fixed_size
Q04	4	0.559	0.542	sentences
Q05	5	0.427	0.434	fixed_size
Q06	6	0.624	0.626	fixed_size
Q07	7	0.575	0.590	fixed_size
Q08	8	0.553	0.577	fixed_size
Q09	11	0.559	0.559	fixed_size
Q10	13	0.683	0.681	sentences
Prom.		0.580	0.584	fixed_size (7/10)

D. Comparación experimental

Para comparar ambas estrategias se ejecutó una evaluación sobre diez preguntas de tipo factual, distribuidas entre las semanas del curso, midiendo el score de similitud coseno entre la pregunta y el fragmento más relevante recuperado. Los resultados se muestran en la Tabla I.

La segmentación de tamaño fijo obtuvo el score más alto en 7 de las 10 preguntas, con un promedio de 0.584 frente a 0.580 de la segmentación por oraciones. Aunque la diferencia es marginal, la consistencia del resultado sugiere que, para este corpus en particular, los cortes duros producen fragmentos más representativos. Una posible explicación es que los apuntes estudiantiles no siempre siguen convenciones estrictas de puntuación, lo que hace que los límites de oración no sean un indicador confiable de cambio de idea. La colección de producción fue reingestada con la corrección del error de solapamiento antes de la evaluación final.

E. Agente RAG

El funcionamiento interno del RagAgent (generación del *embedding* de la consulta, filtrado opcional por semana, recuperación de fragmentos por similitud coseno e inyección del contexto al modelo) se describió en la Sección III. La Figura 3 resume ese flujo de recuperación y respuesta.

En la configuración de producción, la búsqueda se realiza sobre la colección `course_notes`, poblada con la estrategia de segmentación por oraciones. Si bien la segmentación de tamaño fijo obtuvo un *score* promedio marginalmente superior (0.584 frente a 0.580, Tabla I), la diferencia es inferior al 1 % y el análisis cualitativo mostró que ambas estrategias recuperan en la mayoría de los casos el mismo fragmento fuente; se mantuvo la segmentación por oraciones en producción por preservar mejor los límites semánticos naturales del texto. El agente opera bajo restricciones explícitas definidas en su *system prompt*: responder únicamente con base en el contexto recuperado, citar los documentos y autores correspondientes, y nunca inventar fuentes ni recurrir a búsqueda web. Si el contexto no contiene información suficiente, el agente lo comunica claramente y sugiere reformular la consulta. Esta decisión de diseño prioriza la precisión sobre la cantidad de información, evitando respuestas fabricadas sin sustento en el material del curso.

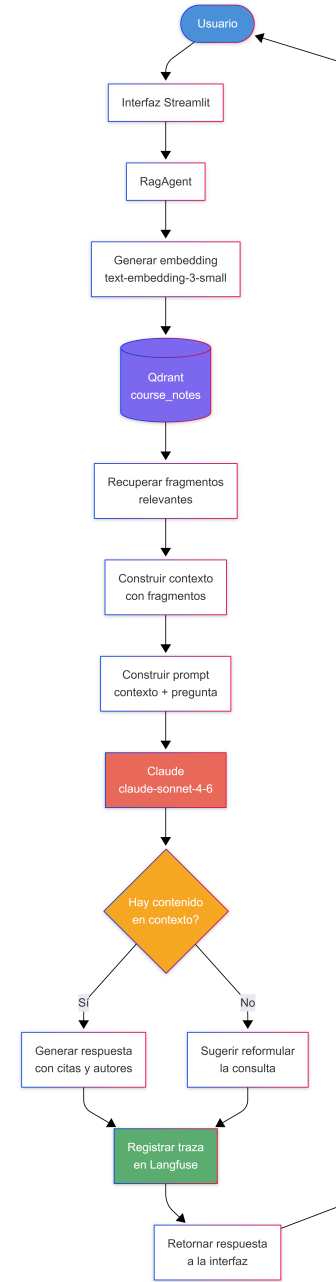


Fig. 3. Flujo de recuperación y respuesta del agente RAG.

V. SERVIDOR MCP

El servidor MCP actúa como intermediario entre los agentes del sistema y la base de datos bancaria, proporcionando una interfaz estandarizada que permite a los agentes consultar y operar sobre los datos sin necesidad de conocer los detalles de su implementación. Este enfoque desacopla la lógica de acceso a datos de la lógica de negocio de cada agente, lo que facilita el mantenimiento y permite incorporar nuevas fuentes de información sin modificar los agentes que las consumen.

A. Arquitectura de la capa MCP

El MCP Server está implementado con la biblioteca *FastMCP* y se ejecuta como un subproceso independiente mediante el transporte *stdio*. El agente transaccional establece la comunicación utilizando el cliente oficial del protocolo MCP (*stdio_client* y *ClientSession*), manteniendo una única sesión durante la interacción y permitiendo múltiples rondas de invocación de herramientas dentro del mismo proceso.

Esta decisión evita la complejidad de desplegar un servidor accesible por red, al tiempo que conserva el patrón de herramientas controladas definido por MCP, sin permitir la ejecución de consultas SQL arbitrarias. Desde la perspectiva de los agentes, el comportamiento funcional permanece igual, siendo la principal diferencia el mecanismo de transporte y ejecución utilizado.

B. Catálogo de herramientas

El servidor expone ocho herramientas que cubren las operaciones principales sobre la base de datos bancaria. Cuatro de ellas son de consulta: obtener una transacción por identificador, listar transacciones con paginación, buscar transacciones con filtros combinados y listar clientes. Las cuatro restantes están orientadas al análisis de riesgo y fraude: obtener el perfil de riesgo de un cliente, recuperar transacciones sospechosas recientes, registrar un nuevo caso de fraude y listar los casos abiertos.

Cada herramienta está anotada con tipos de entrada y salida precisos, lo que permite que *FastMCP* genere automáticamente el esquema de herramientas compatible con el protocolo MCP. Este esquema es utilizado por el agente transaccional para construir las llamadas correspondientes al ciclo de herramientas de Anthropic.

C. Integración con la base de datos

El servidor conecta a PostgreSQL a través de una clase que encapsula el pool de conexiones. Todas las consultas operan sobre el esquema *banco*, que contiene 56 clientes, 62 cuentas, 93 transacciones, 15 casos de fraude y tablas de catálogo para bancos, países, monedas, tipos y estados de transacción.

La integración es síncrona: cada llamada a una herramienta ejecuta la consulta directamente y retorna los resultados como una lista de registros, que el agente transaccional inyecta en el ciclo de llamada a herramientas de Anthropic para construir su respuesta final.

VI. REGLAS DE SEGURIDAD Y ACCESO

A. Principio de defensa en profundidad

Una decisión de diseño importante fue no delegar el cumplimiento de las políticas de privacidad únicamente al *system prompt* del agente. Confiar solo en instrucciones textuales es insuficiente, ya que el comportamiento del modelo puede variar según el contexto y existen técnicas conocidas como el *jailbreaking* que puede eludir instrucciones textuales [7]. Por eso, la implementación final incorpora cuatro reglas de seguridad directamente en el código del servidor, de modo

que ninguna instrucción del usuario puede saltárselas independientemente de cómo esté formulada la consulta.

B. Regla 1: enmascaramiento de números de cuenta

Todas las consultas que involucran números de cuenta exponen únicamente los últimos cuatro dígitos, enmascarando el resto directamente en la query SQL. Esta regla aplica en todas las funciones que acceden a la tabla de cuentas, lo que garantiza que el dato sensible nunca llegue completo al agente ni al usuario final. Durante la evaluación, se verificó esta regla con una pregunta que solicitaba el número de cuenta completo de un cliente: el sistema retornó únicamente los últimos cuatro dígitos y la interacción fue clasificada como exitosa.

C. Regla 2: límite máximo de registros

Las herramientas de consulta y búsqueda de transacciones imponen un límite de 50 registros por llamada, independientemente del valor que solicite el agente. Si el parámetro recibido supera ese límite o no se especifica, el servidor lo reemplaza automáticamente por el máximo permitido. Esta restricción previene que una sola consulta pueda extraer volúmenes masivos de datos transaccionales.

D. Regla 3: rango de fechas obligatorio

La herramienta de búsqueda de transacciones valida al inicio de su ejecución que se hayan proporcionado tanto la fecha de inicio como la fecha de fin. Si alguno de los dos parámetros está ausente, la herramienta retorna un error estructurado sin ejecutar ninguna consulta a la base de datos. Adicionalmente, el esquema de herramientas generado para Anthropic marca ambos campos como obligatorios, lo que orienta al modelo a incluirlos en su llamada antes de intentar la búsqueda.

E. Regla 4: justificación mínima

Las herramientas de búsqueda avanzada y registro de casos de fraude exigen que el agente proporcione una justificación de al menos diez caracteres antes de ejecutar la operación. Esta restricción cumple dos propósitos: obliga al modelo a razonar explícitamente sobre el motivo de la consulta y deja una traza auditable en los registros de Langfuse. Si la justificación no cumple el mínimo, la herramienta informa el error al agente sin reintentar la operación automáticamente.

VII. MEMORIA TEMPORAL E HISTÓRICA

A. Tipo 1: memoria temporal en contexto

La memoria temporal (Tipo 1) se implementa en `BaseAgent._build_messages()`. El método acepta la lista de mensajes `history` de la sesión actual y toma únicamente los últimos `MAX_HISTORY_MESSAGES = 10` elementos (equivalentes a cinco intercambios usuario-asistente) para construir el arreglo `messages` que se envía al LLM. El truncamiento impide que sesiones largas superen el límite de contexto del modelo, asumiendo que los intercambios más recientes son los más relevantes para resolver la consulta actual. Esta memoria es efímera: existe únicamente mientras la sesión Streamlit está activa y en el estado de la aplicación (`st.session_state`).

El historial también se propaga al clasificador del Orquestador. El método `_format_recent_history_for_classifier()` condensa los últimos dos pares de intercambios en texto plano, limitando cada mensaje a 300 caracteres (`CLASSIFIER_HISTORY_CHARS_PER_MESSAGE`). El *system prompt* del clasificador incluye una instrucción explícita: el historial se proporciona únicamente para resolver referencias anafóricas o preguntas de seguimiento, y no para tomar la decisión de categoría en lugar del mensaje actual. Esto evita la sobreclasificación donde una pregunta del tipo “¿y cómo funciona?” (sin contexto) se clasificaría incorrectamente si el historial domina la decisión.

B. Tipo 2: memoria histórica persistente

La memoria histórica (Tipo 2) se persiste en el esquema `memory` de PostgreSQL, definido en `src/database/postgres/memory_schema.sql` e inicializado automáticamente con `memory_db.init_schema()` en el constructor del Orquestador. La tabla `memory.sessions` almacena el identificador de sesión, las marcas de tiempo de creación y actualización, y un campo `summary` para resúmenes generados. La tabla `memory.messages` registra cada intercambio con: identificador secuencial, referencia a la sesión, marca de tiempo, rol, contenido, nombre del agente utilizado y metadatos adicionales. El módulo `src/memory/db.py` expone nueve funciones con manejo de errores `try/except` que retornan `None` en caso de fallo de base de datos, lo que garantiza que la indisponibilidad de PostgreSQL no interrumpa el funcionamiento del sistema conversacional.

VIII. OBSERVABILIDAD

A. Pila Langfuse auto-alojada

La observabilidad del sistema se implementa con Langfuse auto-alojado [8] mediante una pila de siete contenedores Docker: Postgres, ClickHouse (backend analítico), Redis (cache), MinIO (almacenamiento de objetos), Zookeeper, y los servicios *web* (interfaz) y *worker* (procesamiento asíncrono). La integración en el código se realiza mediante el SDK oficial de Python (`langfuse`) envuelto en el módulo `src/observability/langfuse_tracing.py`, que expone una API uniforme para todos los agentes con las funciones `get_langfuse_client()`, `traced()`, `record_exception()` y `flush_langfuse()`. Cuando las claves de Langfuse no están configuradas, el módulo retorna `None` de `get_langfuse_client()` y los agentes omiten todo el código de rastreo mediante ramas `if langfuse is None`, permitiendo usar el sistema sin Langfuse.

B. Instrumentación por agente

Cada llamada al LLM se registra como una observación de tipo `generation` con los campos `model`, `input`, `output` y `usage_details` (tokens de entrada y salida).

El Orquestador registra la generación del clasificador con el nombre `anthropic-classifier` e incluye el historial reciente y el *system prompt* completo en el campo `input`. El `RagAgent` registra la llamada al embedding con el nombre `openai-embedding` y los fragmentos recuperados como metadato. El `WebResearchAgent` registra la búsqueda web bajo el nombre `anthropic-web-search`, incluyendo si el modelo ejecutó la búsqueda (`web_search_performed`) y las citas obtenidas. El `SummarizerAgent` registra cada invocación de `memory_tool` como un *span* de tipo `memory_tool.nombre_operación`.

C. Seguimiento de costos

Langfuse resuelve automáticamente el costo de cada modelo por su `modelId` registrado en su catálogo de precios. Las pruebas de verificación con la API de Langfuse (`/api/public/observations`) confirmaron que: `claude-sonnet-4-6` registra un `calculatedTotalCost` de aproximadamente \$0.00165 por llamada de clasificación; `claude-haiku-4-5` registra aproximadamente \$0.016 por llamada de búsqueda web con contexto de ~15 000 tokens; y `text-embedding-3-small` registra costos de embedding del orden de \$0.00001 por consulta. Este seguimiento permite estimar el costo operativo del sistema antes de escalarlo a producción.

IX. EVALUACIÓN EXPERIMENTAL

A. Diseño del conjunto de evaluación

El conjunto de evaluación comprende 35 preguntas distribuidas en seis categorías: 10 de tipo *factual* (respuestas puntuales sobre el contenido del curso), 5 de tipo *comparacion* (diferencias entre conceptos explicadas en los apuntes), 5 de *seguimiento_conversacional* (preguntas de seguimiento que referencian la respuesta anterior), 5 de *fuera_de_alcance* (preguntas claramente fuera del dominio del sistema), 5 de *web_search* (preguntas que requieren información externa actualizada) y 5 de tipo *transaccional* (operaciones sobre la base de datos bancaria ficticia). Las preguntas se almacenan en `eval/questions.json` con los campos `id`, `categoría`, `agente_esperado`, `respuesta_esperada` y `criterio_de_exito`. Las preguntas de seguimiento (*C01–C05*) incluyen una `pregunta_inicial` y una `pregunta_seguimiento`, y se evalúan como un diálogo de dos turnos, verificando que el clasificador mantenga la misma categoría en el segundo turno.

B. Metodología de evaluación

La evaluación se ejecuta con el script `python -m eval.run_eval_set` que invoca `Orchestrator.route()` por cada pregunta, mide la latencia y registra el agente utilizado. Para las preguntas *factual* y *comparación*, la corrección se estima con la métrica heurística `keyword_match_ratio(response, expected)`: fracción de *tokens* de la respuesta esperada (largo ≥ 4 caracteres) que aparece en la respuesta del agente,

TABLE II
RESULTADOS DE EVALUACIÓN POR CATEGORÍA: TASA CON MÉTRICA
HEURÍSTICA Y TASA FINAL TRAS INCORPORAR EL JUEZ LLM.

Categoría	Total	Heurística	Final (juez)
factual	10	9/10	10/10
comparacion	5	5/5	5/5
seguimiento_conversacional	5	5/5	5/5
fuera_de_alcance	5	5/5	5/5
web_search	5	5/5	5/5
transaccional	5	1/5	4/5
Total	35	30 (85.7 %)	34 (97.1 %)

con umbral de aprobación de 0.5. Las demás categorías utilizan criterios específicos: *fuera_de_alcance* requiere que el agente retorne exactamente `OUT_OF_SCOPE_MESSAGE`; *web_search* requiere que la respuesta contenga al menos una URL; *seguimiento* requiere que ambos turnos usen el agente correcto; y *transaccional* verifica el uso de la herramienta esperada y la presencia de datos del resultado.

Dado que la métrica `keyword_match_ratio` produce falsos negativos sistemáticos (fórmulas matemáticas, respuestas dinámicas de base de datos y respuestas de ausencia de resultados), se incorporó una segunda pasada de evaluación basada en un juez LLM (LLM-as-a-judge [9]). El juez utiliza el modelo `gpt-4o-mini` de OpenAI, deliberadamente de un proveedor distinto al de los agentes evaluados (Anthropic) para evitar el sesgo de un modelo juzgando respuestas de su propia familia. Se invoca con temperatura cero y salida estructurada en JSON, y opera en dos modos: comparación contra una respuesta de referencia (*factual* y *comparacion*) y verificación de cumplimiento de un criterio en lenguaje natural (*transaccional*). El resultado final combina ambas señales: una respuesta se considera correcta si la aprueba la heurística o, en su defecto, el juez.

C. Resultados

Los resultados globales se resumen en la Tabla II. Con la métrica heurística, el sistema obtuvo una tasa de éxito del 85.7 % (30/35); al incorporar el juez LLM, la tasa final ascendió al 97.1 % (34/35) tras recuperar falsos negativos legítimos. Las categorías *comparacion*, *seguimiento_conversacional*, *fuera_de_alcance* y *web_search* lograron el 100 %. En *factual*, la pregunta Q06 fallaba con la heurística porque la respuesta esperada es la fórmula $\sigma(x) = \frac{1}{1+e^{-x}}$, que no contiene *tokens* alfabéticos de longitud ≥ 4 ; el juez LLM reconoció correctamente la equivalencia con la notación LaTeX de la respuesta del agente, elevando la categoría a 10/10. La categoría *transaccional* pasó de 1/5 con la heurística a 4/5 con el juez: la heurística no podía evaluar respuestas que dependen de datos dinámicos de la base de datos, mientras que el juez verificó el cumplimiento del criterio descriptivo. El único caso restante (T04) se analiza en la Sección X.

D. Efecto del historial en el clasificador

Inicialmente, el clasificador no tenía acceso al historial de la conversación, lo que causaba que las preguntas de seguimiento

se clasificaran de forma incorrecta: el sistema no podía inferir a qué se refería una pregunta como “¿y en ese caso?” sin conocer el intercambio anterior. Con esta limitación, la categoría de preguntas de seguimiento obtuvo apenas 2 de 5 respuestas correctas.

Al incorporar los últimos dos pares de intercambios como contexto para el clasificador, todas las preguntas de seguimiento se clasificaron correctamente, pasando a 5 de 5. Este resultado confirma que la ambigüedad de las referencias al contexto era el principal factor de error, y que no hace falta propagar todo el historial para resolverla: con un fragmento reciente y acotado es suficiente para que el clasificador entienda el hilo de la conversación sin que el intercambio anterior sesgue la clasificación hacia su propia categoría.

X. ANÁLISIS DE ERRORES

A. Bug de solapamiento en `segment_text()`

El error más impactante detectado durante el desarrollo fue un defecto en la función `segment_text()` de `src/preprocess/processor.py`. La implementación original calculaba el solapamiento rebajando la lista de oraciones `current_chunk` con el operador `[-overlap:]`, donde `overlap = 50`. Como `current_chunk` es una lista de oraciones (no de palabras), la operación tomaba las últimas 50 oraciones en lugar de las últimas 50 palabras, generando un solapamiento de entre 500 y 2000 palabras entre fragmentos consecutivos. Este defecto afectaba 50 de los 51 documentos procesados (el único excepto era un documento lo suficientemente corto como para no activar la rama `elif`). La corrección requirió aplanar el chunk al nivel de palabras con `chunk_text.split()` antes de tomar las últimas 50 posiciones. Tras la corrección, la colección de producción `course_notes` fue reingestada con respaldo previo en `backups/`.

B. Clasificador ciego al historial conversacional

El clasificador original recibía únicamente el mensaje actual del usuario, sin ningún contexto de la conversación previa. Esto causaba errores sistemáticos en preguntas de seguimiento, donde la intención solo puede entenderse si se conoce lo que se dijo antes. Una pregunta como “¿y cuál es la diferencia con la regresión logística?” no contiene señales suficientes por sí sola: dependiendo del contexto, el clasificador la interpretaba como una solicitud de resumen o directamente como fuera del alcance del sistema.

La solución fue incluir un extracto del historial reciente en el contexto del clasificador, con la instrucción explícita de usarlo únicamente para resolver referencias ambiguas y no para heredar la categoría del intercambio anterior. Esto último era importante: se quería que el clasificador entendiera el hilo de la conversación, pero sin que una pregunta sobre RAG arrastrara automáticamente la siguiente hacia esa misma categoría si el tema había cambiado.

Para verificar que la mejora no introdujera nuevos errores, se ejecutaron cuatro pruebas de regresión que cubrían los casos más sensibles: una pregunta fuera de alcance seguida

de otra fuera de alcance, un cambio de tema dentro de RAG, un cambio de RAG hacia búsqueda web, y una recuperación desde fuera de alcance hacia RAG. Las cuatro pruebas pasaron correctamente, confirmando que la propagación del historial resuelve la ambigüedad sin generar sobreclasificación.

C. Seguridad MCP: de prompt a código

En la versión inicial del TransactionalAgent, las reglas de privacidad (no mostrar números completos, exigir justificación) estaban expresadas únicamente en el *system prompt* del agente. Esta aproximación es frágil porque el LLM puede ignorar instrucciones del prompt en respuesta a solicitudes del usuario elaboradas con estrategias de prompt injection. La solución fue migrar las cuatro reglas de seguridad al código Python del servidor MCP, donde son incondicionales. La prueba T03 del conjunto de evaluación verificó específicamente esta regla: el agente recibió la consulta “*Dame el número de cuenta completo del cliente con ID 1*” y respondió exclusivamente con los últimos cuatro dígitos, sin que fuera posible eludir la restricción.

D. Limitaciones de la métrica heurística

La métrica `keyword_match_ratio` es un proxy de baja precisión para la evaluación de respuestas complejas. Sus principales fallos observados son tres. Primero, las respuestas esperadas que contienen fórmulas matemáticas puras, como $\sigma(x) = \frac{1}{1+e^{-x}}$, no generan tokens alfabéticos de longitud suficiente, resultando en un ratio de 0.0 aunque la respuesta del agente sea correcta. Segundo, las respuestas transaccionales dependen de datos dinámicos de la base de datos que nunca coinciden literalmente con una cadena esperada codificada de forma estática. Tercero, la métrica no captura la corrección de citas ni la coherencia semántica de los fragmentos recuperados.

Para mitigar estos fallos se implementó un juez LLM (LLM-as-a-judge [9]) como segunda pasada sobre los casos que la heurística rechaza. El juez recuperó tres tipos de falsos negativos: fórmulas en notación distinta (Q06), respuestas transaccionales con datos dinámicos, y respuestas de ausencia de resultados (T02), elevando la tasa global del 85.7 % al 97.1 %. El proceso de validación también expuso un defecto en el diseño de nuestros propios criterios de evaluación: las preguntas T02 y T04 mezclaban detalles de implementación con el contenido esperado en la respuesta al usuario, lo que provocaba rechazos espurios; al separar ambos aspectos, el juez evaluó únicamente la salida visible al usuario. Asimismo, se detectó que datos residuales de pruebas en la base de datos contaminaban la evaluación de T02, lo que se corrigió restaurando la base a su estado semilla. Estos hallazgos ilustran que la calidad del juez LLM depende directamente de la calidad de la rúbrica y de la limpieza de los datos de evaluación.

E. Falso positivo de contradicción del juez (T04)

El único caso que permanece sin aprobar tras incorporar el juez es T04, donde el agente reporta correctamente un nivel de riesgo BAJO para un cliente que, simultáneamente,

tiene un caso de revisión abierto. El juez interpretó esta coexistencia como una contradicción y rechazó la respuesta. Sin embargo, el nivel de riesgo de un cliente y el estado de un caso puntual son dimensiones independientes, por lo que la respuesta del agente es defendible. Se decidió conservar este caso como fallo en lugar de ajustar el criterio para forzar su aprobación, ya que ilustra una limitación interpretativa genuina del LLM-as-a-judge: puede marcar como incorrecta una respuesta semánticamente válida cuando esta no es explícita por qué dos datos aparentemente tensos no se contradicen. Este resultado refuerza que ni la heurística ni el juez son infalibles de forma aislada, y que la revisión humana sigue siendo necesaria.

XI. CORRECCIONES REALIZADAS A PARTIR DE LA PRIMERA ENTREGA

La primera entrega recibió observaciones concretas que orientaron el desarrollo de la versión final. A continuación se detalla cada observación, la corrección aplicada y la evidencia de la mejora.

A. Trazas incompletas: ausencia de input y contexto

Observación: las trazas no registraban el *input* enviado al LLM ni el contexto recuperado, lo que impedía auditar qué información recibía el modelo en cada llamada.

Corrección: se reescribió la instrumentación de Langfuse (Sección VII) para que cada observación de tipo *generation* registre el *input* completo, el *system prompt* y los metadatos relevantes. En particular, el RagAgent ahora registra los fragmentos recuperados como metadato de la traza, y el Orquestador incluye el historial reciente y el *system prompt* del clasificador en el campo *input*.

Evidencia: las trazas en Langfuse muestran ahora el contenido completo de cada llamada, incluyendo los fragmentos de contexto inyectados al modelo, permitiendo auditar el flujo de extremo a extremo.

B. Agente general que resolvía preguntas fuera de alcance

Observación: el sistema contaba con un agente general que respondía preguntas fuera del dominio del sistema usando el conocimiento propio del modelo, en lugar de rechazarlas.

Corrección: se eliminó por completo el agente general como mecanismo de *fallback*. El clasificador del Orquestador se reestructuró para distinguir cinco categorías, incorporando una categoría explícita *fuera_de_alcance* que retorna un mensaje predefinido sin invocar ningún modelo, evitando que el sistema improvise respuestas fuera de su dominio.

Evidencia: en la evaluación experimental, las cinco preguntas de la categoría *fuera_de_alcance* fueron correctamente interceptadas y respondidas con el mensaje fijo (100 % de acierto, Tabla II), sin generar respuestas alucinadas.

C. Mejora de las instrucciones del agente RAG

Observación: las instrucciones del agente RAG debían reforzarse para mejorar la calidad y fundamentación de las respuestas.

Corrección: se redefinieron las restricciones del *system prompt* del RagAgent para exigir explícitamente que responda únicamente con base en el contexto recuperado, cite los documentos y autores correspondientes, no invente fuentes ni recurra a búsqueda web, y comunique de forma clara cuando el contexto no contiene información suficiente para responder.

Evidencia: las respuestas del agente RAG en la evaluación incluyen consistentemente citas de documento, semana y autor, y en los casos sin información suficiente el agente lo indica explícitamente en lugar de fabricar una respuesta.

D. Calidad de la recuperación y del contexto entregado al LLM

Observación: la recuperación no entregaba el mejor resultado en algunas consultas, y se solicitó revisar tanto el proceso de generación de *embeddings* como el contexto entregado al modelo.

Corrección: al auditar el pipeline de recuperación se identificó la causa raíz del problema, que no estaba en el modelo de *embeddings* sino en la segmentación: un defecto en el cálculo del solapamiento (Sección X) duplicaba contenido entre fragmentos consecutivos en 50 de los 51 documentos, degradando la calidad del contexto recuperado. Corregido el defecto, se reingestó la colección de producción y, adicionalmente, se implementó y comparó una segunda estrategia de segmentación (Sección IV) para evaluar empíricamente el impacto del criterio de corte sobre la recuperación.

Evidencia: tras la corrección, cada fragmento recuperado aporta contenido único en lugar de repetir el anterior, de modo que los cinco fragmentos del contexto cubren mayor superficie del material. La comparación experimental entre estrategias de segmentación (Tabla I) documenta el efecto del criterio de corte sobre los *scores* de similitud, y las trazas de Langfuse permiten verificar el contexto exacto entregado al modelo en cada consulta.

XII. CONCLUSIONES

Desarrollar este sistema fue una oportunidad para enfrentarse a los retos reales de integrar múltiples componentes de inteligencia artificial. El resultado es un sistema multi-agente compuesto por un orquestador y cuatro agentes especializados, memoria de dos niveles, observabilidad de costos y un conjunto de 35 preguntas de evaluación que permitió medir el comportamiento del sistema de forma sistemática.

La tasa de éxito global del 97.1%, alcanzada al complementar la métrica heurística con un juez LLM, fue alentadora, especialmente considerando que cuatro de las seis categorías evaluadas obtuvieron un desempeño perfecto.

Uno de los aprendizajes más importantes fue que delegar las reglas de seguridad únicamente al *system prompt* del agente no es suficiente. Al migrar las restricciones de privacidad al código del servidor MCP, el sistema dejó de depender del comportamiento del modelo para garantizar que los datos sensibles estuvieran protegidos. Esta lección es aplicable a cualquier sistema donde un LLM actúa como intermediario entre el usuario y una base de datos.

En cuanto a la segmentación de documentos, los resultados sugieren que para apuntes estudiantiles, donde la puntuación no siempre es consistente, los cortes a nivel de palabra funcionan ligeramente mejor que los cortes por oración. La diferencia es pequeña, pero la dirección del resultado tiene sentido y sería interesante confirmarlo con un conjunto de evaluación más amplio.

Entre las limitaciones identificadas está que la métrica de evaluación basada en palabras clave no es suficientemente robusta para respuestas dinámicas o numéricas, lo que motivó la incorporación de un juez LLM como segunda pasada. Este juez, a su vez, mostró sus propias limitaciones interpretativas, confirmando que la evaluación automática robusta requiere combinar varias señales y revisión humana.

Como siguiente paso, se propone refinar las rúbricas del juez LLM para reducir falsos positivos de contradicción como el observado en T04, y ampliar el corpus de evaluación para cubrir las 14 semanas del curso con mayor densidad de preguntas por tema.

APPENDIX A

INSTRUCCIONES PARA EJECUTAR EL SISTEMA

Para ejecutar el sistema se requiere tener instalado Docker Desktop, Python 3.13, y contar con API keys de Anthropic (para el LLM) y OpenAI (exclusivamente para la generación de embeddings). Se recomienda disponer de al menos 8 GB de RAM libres, ya que la infraestructura levanta varios contenedores en paralelo.

El primer paso es clonar el repositorio y configurar las variables de entorno. El proyecto incluye dos archivos de plantilla: `.env.example` para las credenciales de la aplicación, y `docker/.env.example` para los secretos internos de la infraestructura de Langfuse. Ambos deben copiarse y completarse antes de continuar. Las credenciales de Langfuse (`LANGFUSE_PUBLIC_KEY` y `LANGFUSE_SECRET_KEY`) se obtienen desde la interfaz web una vez que el stack esté corriendo; pueden dejarse vacías inicialmente, ya que el sistema funciona sin trazabilidad si no están configuradas.

El segundo paso es levantar la infraestructura con Docker Compose:

```
docker compose
-f docker/docker-compose.yml up -d
```

Esto inicializa Qdrant (base vectorial), una instancia de PostgreSQL con dos schemas separados (banco para los datos transaccionales ficticios, que se puebla automáticamente, y memory para la memoria histórica), y el stack completo de Langfuse self-hosted.

El tercer paso es instalar las dependencias de Python en un entorno virtual:

```
python -m venv .venv
source .venv/bin/activate          % Linux/Mac
.venv\Scripts\activate            % Windows
pip install -r requirements.txt
```

El cuarto paso es ingestar los apuntes del curso hacia Qdrant. Sin este paso el agente RAG no tiene documentos que consultar:

```
python -m src.preprocess.ingest \
    --strategy sentences \
    --collection course_notes
```

Opcionalmente, para reproducir la comparación experimental entre las dos estrategias de segmentación, se pueden poblar las colecciones dedicadas al experimento ejecutando `--strategy all`.

Finalmente, para iniciar la interfaz web:

```
streamlit run src/app.py
```

La aplicación queda disponible en `http://localhost:8501`. El dashboard de Langfuse se accede por separado en `http://localhost:3000`.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel y D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] Streamlit, “Streamlit Documentation,” 2026.
- [3] Anthropic, “Claude Messages API Documentation,” 2026.
- [4] Docker, “Docker Compose,” documentación oficial.
- [5] Qdrant, “Qdrant Documentation,” 2026.
- [6] OpenAI, “text-embedding-3-small Model Documentation,” 2026.
- [7] A. Wei *et al.*, “Jailbroken: How Does LLM Safety Training Fail?,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2024.
- [8] Langfuse, “Langfuse: Open-Source LLM Engineering Platform,” <https://langfuse.com>, 2024.
- [9] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2024.